✅ Lab: Walkthrough of Virtualized GPU Setup via NGC or Simulated Lab

🔍 Goal:

Learn how to **provision and run a virtualized GPU instance** using **NVIDIA NGC** or simulate vGPU setup using **local tools or an online sandbox**, observing how multiple workloads can run on a single GPU with resource isolation.

🧪 Option A: Use NVIDIA LaunchPad (Recommended – if access is available)

NVIDIA LaunchPad provides access to real infrastructure (A100/H100 with vGPU and MIG enabled).

🔧 Step-by-Step Instructions:

🔹 Step 1: Sign up and access NVIDIA LaunchPad

1. Go to: https://www.nvidia.com/en-us/ngc/launchpad
2. Sign in with your NVIDIA Developer account (free to create).
3. Search for the “Virtualized AI Workload Lab” or “vGPU Evaluation Lab”.
4. Apply to access the lab. Approval takes 1–2 business days.

🔹 Step 2: Launch your environment

Once approved:

1. Launch your environment with **vSphere** or **NVIDIA AI Enterprise image**.
2. Select an A100 or H100 instance with **MIG or vGPU** enabled.
3. Open your **vSphere or terminal dashboard**.

🔹 Step 3: Provision virtual GPUs

1. Use vSphere to create 2–3 **virtual machines (VMs)**.
2. Assign a **vGPU profile** to each VM (e.g., 1g.5gb, 2g.10gb).
3. Boot each VM with the **NVIDIA AI Enterprise stack** (Ubuntu-based image).

🔹 Step 4: Run AI workloads on each vGPU

1. On each VM, clone a simple AI workload repo:

```
git clone https://github.com/NVIDIA/DeepLearningExamples.git
cd DeepLearningExamples/PyTorch/Classification/ResNet
```

2. Install dependencies and run training:

```
pip install -r requirements.txt
python main.py --model resnet18 --batch-size 64
```

3. Repeat on all VMs in parallel.

4. Use `nvidia-smi` to monitor each workload and GPU partitioning in action.

🔷 **Step 5: Observe performance isolation**

1. Run `nvidia-smi` or DCGM tools on the host:

```
watch -n 1 nvidia-smi
```

2. Observe how memory and compute resources are divided between VMs.

3. Optionally stress test one VM and observe how it **does not affect** the others.

💻 Option B: Simulated Lab (Colab or Local with Docker)

If LaunchPad access is unavailable, simulate a similar concept using **Docker containers** and software emulation.

🔧 Step-by-Step Instructions:

🔷 **Step 1: Set up environment**

On a local Linux machine with NVIDIA GPU:

1. Install **Docker** and **NVIDIA Container Toolkit**:

```
sudo apt install docker.io
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | sudo apt-key add -
curl -s -L https://nvidia.github.io/nvidia-docker/$distribution/nvidia-docker.list |
sudo tee /etc/apt/sources.list.d/nvidia-docker.list
sudo apt update && sudo apt install -y nvidia-docker2
sudo systemctl restart docker
```

🔷 **Step 2: Simulate virtual GPU environments using containers**

1. Pull base image:

```
docker pull nvcr.io/nvidia/pytorch:23.06-py3
```

2. Launch two containers simulating two users:

```
docker run --rm -it --gpus '"device=0"' --name user1 nvcr.io/nvidia/pytorch:23.06-py3
docker run --rm -it --gpus '"device=0"' --name user2 nvcr.io/nvidia/pytorch:23.06-py3
```

3. Inside each, run a sample training loop:

```
import torch
model = torch.nn.Linear(1000, 1000).cuda()
data = torch.randn(64, 1000).cuda()
for _ in range(1000):
    out = model(data)
```

4. Use `nvidia-smi` outside the container to monitor utilization.

🔷 **Step 3: Analyze and reflect**

- Observe how **Docker containers emulate workload isolation**.
- Discuss how in real vGPU setups, the GPU is **partitioned at the hardware level** with stronger enforcement.

📘 Learning Outcomes:

By the end of this lab, learners should be able to:

- Understand how vGPUs and MIG enable GPU sharing.
- Simulate or run real workloads in virtualized environments.
- Monitor GPU usage across multiple concurrent AI jobs.
- Identify benefits of workload isolation and resource utilization.
